TP final

Groupe 3

Layla & Helena

DOCKER & DOCKER COMPOSE

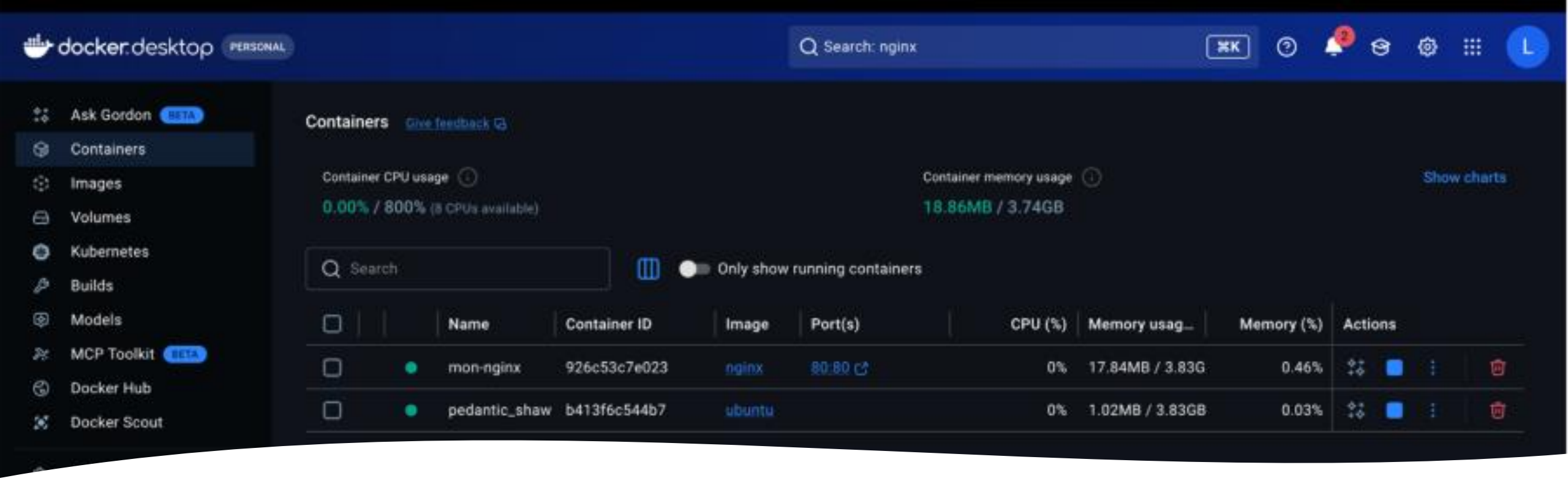
NIVEAU F

Contexte

Avant de sécuriser quoi que ce soit, il faut comprendre les bases. Vous allez lancer votre premier container Docker et explorer son isolation.

Missions

- Vérifiez que Docker est bien installé sur votre machine.
- Lancez un container Ubuntu interactif : explorez son système de fichiers.
- Lancez un serveur web Nginx et accédez-y depuis votre navigateur.
- Listez les containers en cours d'exécution, puis stoppez-les proprement.
- Observez ce qui se passe quand vous supprimez un container : les données sont-elles conservées ?



Lancer ubuntu et nginx

Commande pour lancer un contenaire interactif : **docker run –it ubuntu**

Pour lancer un serveur nginx : **docker run -d -p 80:80 --name mon-nginx nginx**

```
docker run [OPTIONS] IMAGE[:TAG|@DIGEST] [COMMAND] [ARG...]
```

Explications commande **docker run -d -p 80:80 --name mon-nginx nginx** :

- Ça vient chercher l'image nginx dans docker hub
- **-d** : en arrière-plan pour qu'on puisse aller dans un autre contenaire pdt que notre nginx soit en cours
- **80:80** pour mapper le port 80 sur le port 80 de mon ordi et que je puisse y accéder par la suite avec mon navigateur.

Explorer son système de fichier

Explorer son système de fichier avec la commande:
docker exec -it boring_ganguly bash

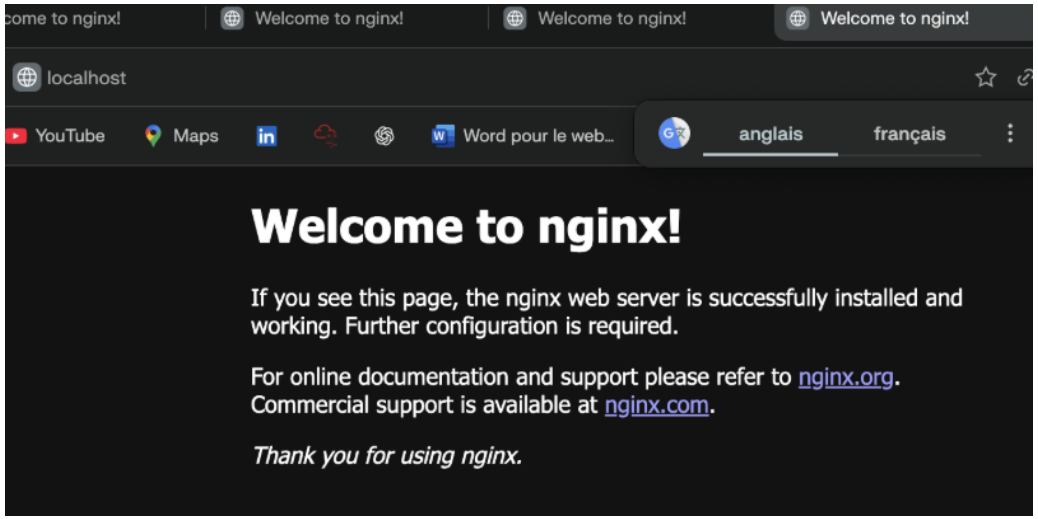
```
layla@Ordinateur-portable-de-Layla ~ % docker exec -it boring_ganguly bash
root@003917b49908:/# ls
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```

```
root@003917b49908:/etc# ls
alternatives            dpkg          host.conf     legal          opt          rc2.d        shadow       system
apt                     e2scrub.conf  hostname      libaudit.conf  os-release   rc3.d        shadow-      termin
bash.bashrc             environment   hosts         login.defs     pam.conf     rc4.d        shells       update
bindresvport.blacklist  fstab         init.d        logrotate.d    pam.d        rc5.d        skel         xattr.
cloud                   gai.conf      issue         lsb-release    passwd       rc6.d        subgid
cron.d                  gnutls        issue.net     machine-id     passwd-      rcS.d        subgid-
cron.daily              group         kernel        mke2fs.conf    profile      resolv.conf  subuid
debconf.conf            group-        ld.so.cache   mtab           profile.d    rmt          subuid-
debian_version          gshadow       ld.so.conf    networks       rc0.d        security     sysctl.conf
default                 gshadow-      ld.so.conf.d  nsswitch.conf  rc1.d        selinux      sysctl.d
root@003917b49908:/etc#
```

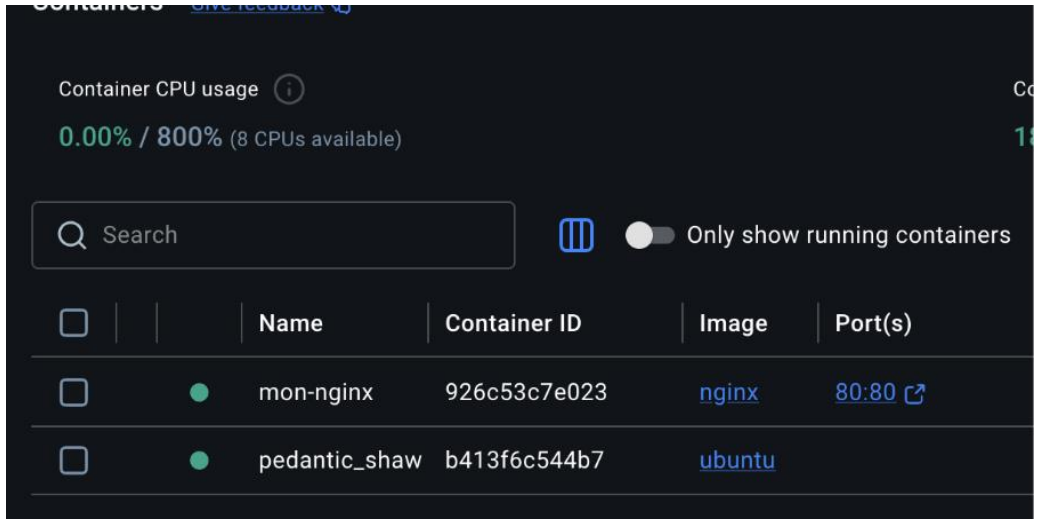
Puis on navigue dans le contenaire avec les commandes **cd** (change directory) **ls**, a l'occurence dans /etc, qui est le fichier de configuration de système d'ubuntu.

Les contenaires en cours d’execution :

- mon-ngninx (nginx)
- pedantic_shaw (ubuntu)

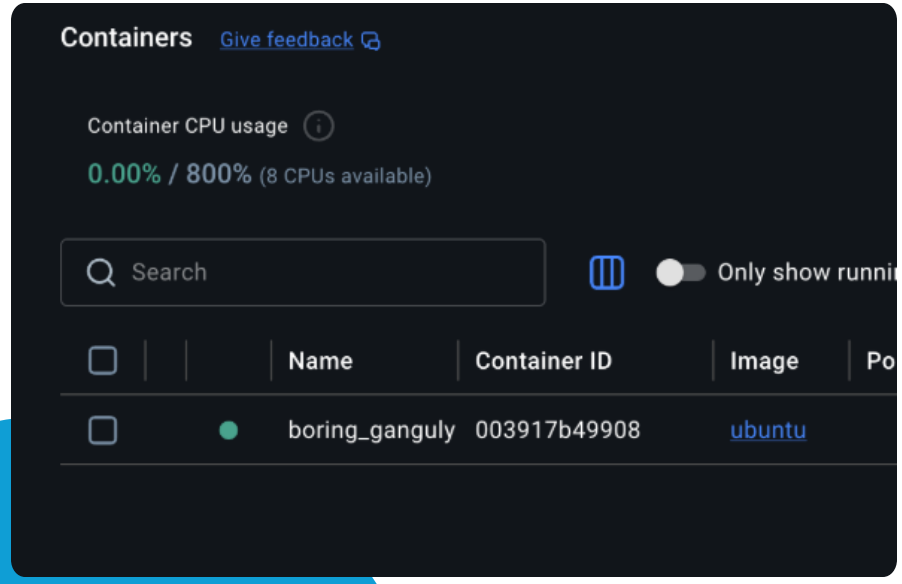


Je vérifie dans mon navigateur que Nginx est bien lancé sur port **8080:80**



```
layla@Ordinateur-portable-de-Layla ~ % docker stop mon-nginx
mon-nginx
layla@Ordinateur-portable-de-Layla ~ % docker rm mon-nginx
mon-nginx
```

Je stoppe mon-nginx et le supprime avec : **docker stop mon-nginx** puis : **docker rm mon-nginx**.



Je constate que le contenaire a disparu, je ne peux plus y accéder.

```
layla@Ordinateur-portable-de-Layla ~ % docker image ls

IMAGE            ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:latest     0236ee02dcbc        258MB         64.1MB
ubuntu:latest    d1e2e92c075e        141MB         30.8MB    U
layla@Ordinateur-portable-de-Layla ~ %
```

L'image néanmoins est tjr dispo. Je vérifie avec docker **image ls.**

Les contenaires et images sont distincts.
Si je veux supprimer l'image, il faudrait que je fasse une commande :
docker rmi image nginx

- **Rmi** : supprimer image
- **Rm** : supprimer contenaire

Questions de reflexion

- **Un conteneur** est un **environnement isolé** qui permet d'exécuter une application avec toutes ses dépendances à partir d'une image Docker. Le contenaire fonctionne sur n'importe quelle machine qui a un docker. Enfaite le but c'est d'avoir un même environnement pour que les supports de tt le monde soient compatibles.

- **Machine virtuelle** : bcp plus lourd, c'est un ordi dans un ordi, c plus gros, plus lent qu'un contenaire. Ça simule un ordi complet, tandis que le contenaire ça sera uniquement pour des images, fichiers, des applis isolées.
  Si je supprime un fichier dans un contenaire, alors le contenaire continue d'exister, même s'il n'a plus de fichiers.

- L'**isolation des contenaires** est importante en cyber car si un contenaire est compromis, il faut que les autres ne soient pas infectés. Les regrouper est une erreur de sécurité.
  Et pour cela, les segmenter est primordial. Il faut explicitement connecter les containeurs au même réseau pr qu'ils puissent communiquer.

NIVEAU E

Contexte

ByteForge avait une application Python Flask qui tournait directement sur le serveur. Votre mission : la

containeriser correctement.

Missions

- Créez le fichier app.py ci-dessus.
- Créez un fichier requirements.txt contenant : flask
- Créez un Dockerfile pour containeriser cette application.
- Construisez l'image : docker build -t byteforge-app .
- Lancez un container depuis cette image et vérifiez que l'application répond.

- Création du fichier app.py .
- Création du fichier requirements.txt contenant : flask
- Création d'un Dockerfile pour containeriser cette application.
- Construction de l'image avec docker build –t byteforge-app
- Lancement du container depuis l'invite de commande avec docker run –d -p 5000:5000 –name mon-app-byteforge byteforge-app

app.py > ...

```python
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
    return '<h1>ByteForge Corp – Système e
@app.route('/status')
def status():
    return {'status': 'ok', 'version': '1.
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```

Dockerfile > ...

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requireme
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\helen>docker run -p 5000:5000 --name mon-app-byteforge-app
docker: 'docker run' requires at least 1 argument

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

See 'docker run --help' for more information

C:\Users\helen>docker run -p 5000:5000 --name mon-app-byteforge byteforge-app
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inste
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [05/Mar/2026 14:41:02] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [05/Mar/2026 14:41:02] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [05/Mar/2026 19:47:16] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [05/Mar/2026 19:47:29] "GET / HTTP/1.1" 400 -
172.17.0.1 - - [05/Mar/2026 19:47:35] "GET /loginMsg.js HTTP/1.1" 404 -
172.17.0.1 - - [05/Mar/2026 19:47:35] "GET /cgi/get.cgi?cmd=home_login HTTP/1.1" 404 -
172.17.0.1 - - [05/Mar/2026 21:18:11] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [05/Mar/2026 21:18:31] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [05/Mar/2026 23:21:44] "GET / HTTP/1.1" 200 -
```

Lancement du container et vérification de l'application

Questions de reflexion

- **Pourquoi utilise-t-on python:3.11-slim plutôt que python:3.11 ?**

On utilise le slim pour avoir une image plus petite et simple car en cybersécurité moins il y'a d'outils inutiles plus la surface d'attaque est réduite.

- **À quoi sert WORKDIR ? Que se passerait-il sans lui ?**

Il définit le dossier par défaut pour toutes les commandes suivantes(run,cmd,copy) sans lui, tout serait copié à la racine

- **Pourquoi copier requirements.txt AVANT le reste du code ?**

C'est pour utiliser le cache des layers Docker.Si je modifier mon code mais pas les dépendances, Docker réutilisera le "pip install" dejà construit, ce qui rend le build beaucoup plus rapide

- **Que fait EXPOSE 5000 ? Est-ce que cela suffit à rendre le port accessible ?**

Cela sert de documentation pour dire que le container écoute sur ce port, pour pouvoir acceder à l'application depuis mon navigateur il faudra que je mappe le port au moment du lancement en utilisant le docker run –p 5000:5000 byteforge- app

Justification technique

/Choix Technique	Pourquoi faire ce choix ?	Conséquence de NE PAS le faire	Faille évitée
WORKDIR /app	Définit un répertoire de travail propre et isolé pour l'application.	Les fichiers sont éparpillés à la racine (/), mélangeant l'app avec les fichiers système.	Divulgation d'informations : Évite qu'une erreur de manipulation n'écrase ou n'expose des fichiers sensibles du système d'exploitation de l'image.
COPY requirements.txt AVANT le code	Permet d'utiliser le cache Docker. Si le code change mais pas les dépendances, Docker ne réinstalle pas tout.	Le build devient extrêmement lent car pip install s'exécute à chaque modification d'une seule ligne de code.	Évite la saturation des ressources réseau et disque lors des phases de construction répétitives.
Utilisation d'une image "slim"	Réduit la taille de l'image au strict nécessaire pour faire tourner Python.	L'image est lourde (plusieurs Go) et contient des outils inutiles (compilateurs, utilitaires réseau).	Réduction de la surface d'attaque : Moins il y a d'outils installés, moins un attaquant a de leviers pour rebondir s'il prend le contrôle du conteneur.
EXPOSE 5000	Documente le port d'écoute de l'application pour les autres développeurs et outils.	Manque de clarté sur le fonctionnement interne ; le mappage des ports devient une devinette.	Principe du moindre privilège : Indique explicitement quel port est nécessaire, évitant d'ouvrir accidentellement des ports non désirés

NIVEAU D

Contexte

L'application de ByteForge tournait en utilisateur root dans le container. C'est l'une des premières

erreurs de sécurité : si le container est compromis, l'attaquant a les droits root.

Missions

- Vérifiez avec qui votre container actuel tourne : docker run byteforge-app whoami
- Modifiez le Dockerfile pour créer un utilisateur dédié non-root.
- Ajoutez l'instruction USER pour basculer sur cet utilisateur.
- Vérifiez que l'application fonctionne toujours.
- Testez de faire un apt-get install dans le container : que se passe-t-il ?

Avec quel utilisateur le conteneur s’exécute-t-il ?

```
layla@Ordinateur-portable-de-Layla td_jeudi % docker run byteforge-app whoami
root
```

Réponse : Il s’exécute avec `root@003917b49908:`

Root a tout les droits, lire, modifier, supprimer n’importe quel fichier, installer et désinstaller des logiciels, changer des comptes utilisateurs, accéder à toutes les ressources matérielles donc des fichiers dans root doivent etre non sensibles.

Dans le cas ici, nous allons rendre plus difficile l’accès à nos fichiers en les déplacant de root a un utilisateur spécifique

On modifie le dockerfile pour changer le RUN :

```
FROM python:3.11-slim
    WORKDIR /app
    RUN groupadd -r appuser && useradd -r -g appuser appuser
    COPY requirements.txt .
    RUN pip install --no-cache-dir -r requirements.txt
    COPY . .
    RUN chown -R appuser:appuser /app
    USER appuser
    EXPOSE 5000
    CMD ["python", "app.py"]
```

Explications :

- groupadd -r → crée un groupe système léger
- useradd -r -g appuser → crée un utilisateur système qui appartient à ce groupe
- Pourquoi ? Docker tourne **par défaut en root**, ici on empêche l'application d'avoir trop de droits.

On vérifie que l’application fonctionne toujours

Commande : docker build –t byteforge-app

```
layla@Ordinateur-portable-de-Layla td_jeudi % docker build -t byteforge-app .
[+] Building 3.8s (13/13) FINISHED
 => [internal] load build definition from dockerfile
 => => transferring dockerfile: 287B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/7] FROM docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278
 => => resolve docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278
 => [internal] load build context
 => => transferring context: 352B
 => CACHED [2/7] WORKDIR /app
 => [3/7] RUN groupadd -r layla && useradd -r -g layla layla
 => [4/7] COPY requirements.txt .
 => [5/7] RUN pip install --no-cache-dir -r requirements.txt
 => [6/7] COPY . .
 => [7/7] RUN chown -R layla:layla /app
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:bf07302ccc38df318b8aba7adc05e79efb132f24fddecf95b593ab132990660a
 => => exporting config sha256:cfced016bb5ae6df9777eba24ce71122812098269af2ca11c4c407ee2d57135c
 => => exporting attestation manifest sha256:cdc37dc42b64d5490c81d4413c5921edb844b89f4f9282bfa22dc9b06679
 => => exporting manifest list sha256:d213601fefb93f00fec1f07699716c0914c7934fdf130c14926a4e2ed3ba7bf4
 => => naming to docker.io/library/byteforge-app:latest
 => => unpacking to docker.io/library/byteforge-app:latest
```

Ca fonctionne bien

```
layla@Ordinateur-portable-de-Layla td_jeudi % docker exec -it boring_ganguly bash
root@003917b49908:/# apt-get install
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@003917b49908:/#
```

1er test root ❗

J'ai dû me connecter à mon containeur ubuntu pour pouvoir executer **apt-get install**.
Commande **docker exec –t boring ganguly bash.**
boring_ganguly : nom

2e test non-root ❗

Étant donné que le conteneur est maintenant exécuté avec un utilisateur **non-root**, la commande **apt-get install** génère une erreur.
En effet, l'installation de paquets nécessite des privilèges administrateur.

```
layla@Ordinateur-portable-de-Layla td_jeudi % docker run -it byteforge-app-secure bash

Unable to find image 'byteforge-app-secure:latest' locally
docker: Error response from daemon: pull access denied for byteforge-app-secure, repository does not exist or ma
y require 'docker login'

Run 'docker run --help' for more information
```

Explications

L'utilisateur créé dans le Dockerfile ne possède pas les mêmes permissions que **root**, il est donc normal qu'il ne puisse pas exécuter cette commande. Pour installer des paquets, il faudrait effectuer cette étape avant de basculer vers l'utilisateur non-root dans le Dockerfile.

Questions de reflexion

Least privilege : C’est une règle de sécurité qui consiste à **donner à un utilisateur, un service ou un processus uniquement les permissions strictement nécessaires à l’exécution de ses tâches**, et rien de plus. C’est pour réduire la surface d’attaque et limiter les dégâts en cas de compromission.

Root dans un container : super-utilisateur avec tous les droits sur le container (fichiers, processus, configurations). Les actions restent confinées au container.

Root sur l’hôte : super-utilisateur de la machine entière, avec accès à tous les fichiers, processus et containers. Une compromission ici permet un contrôle total de la machine et de tous les containers.

root dans le conteneur ≈ root sur l’hôte dans certains cas mal configurés

Principe clé : éviter de lancer des applications en root dans un container pour limiter les risques.

Containeur escape :
En cyber, le containeur escape est l'évènement critique ou l'attaquant arrive a sortir du conteneur et accède au système hôte, soit la machine qui exécute les containeurs.
C'est pour cela qu'on doit isoler les containeurs, car si ces derniers ne sont pas isolés, alors l'attaque se propage très rapidement et risque d'endommager l'ensemble du système et pas qu'une partie. C'est le cas lorsqu'on utilise l'utilisateur root, qui octroie les privilèges maximum dans le conteneur.

NIVEAU C

Contexte

ByteForge avait Flask exposé directement sur Internet. Bonne pratique : placer un reverse proxy (Nginx)

devant. L'utilisateur ne parle qu'à Nginx, qui transmet les requêtes à Flask. Flask n'est plus accessible

de l'extérieur.

Missions

Missions

- Créez un fichier nginx.conf pour proxifier les requêtes vers Flask.
- Créez un docker-compose.yml avec deux services : flask et nginx.
- Configurez un réseau Docker interne pour qu'ils communiquent.
- Le port 5000 de Flask NE DOIT PAS être exposé à l'extérieur.
- Vérifiez que http://localhost fonctionne et que Flask répond via Nginx.

Configuration du Nginx(nginx.conf)

nginx.conf

```
events {}
http {
    server {
        listen 80;
        location / {
            proxy_pass http://flask:5000;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_
        }
    }
}
```

- Mon nginx va recevoir les requetes sur le port 80 et les rediriger vers le service flask

Le fichier docker-compose.yml

Il va permettre la liaison des deux conteneurs dans un réseau privée appelé internal

app.py | requirements.txt | Dockerfile | nginx.conf | docker-compose.yml

docker-compose.yml

```yaml
services:
  flask:
    build: .
    networks:
      - internal
    # PAS de ports: ici ! Flask ne doit pas être accessible de l'extérieur
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - internal
    depends_on:
      - flask
networks:
  internal:
    driver: bridge
```

Lancement et vérification

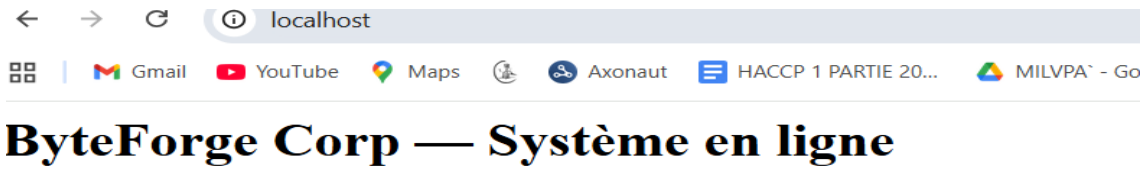
Il se fait en deux étapes:

1er étape: le lancement des services dans le terminal avec **docker-compose up --build**

2e étape: la vérification sur Docker puis sur un navigateur(chrome) avec juste http//:localhost

```
PS C:\Users\helen\Downloads\byteforge_project> docker-compose up --build
#1 [internal] load local bake definitions
#1 reading from stdin 561B 0.2s done
#1 DONE 0.2s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 200B 0.0s done
#2 DONE 0.1s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 ...

#4 [auth] library/python:pull token for registry-1.docker.io
#4 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 1.5s

#5 [internal] load .dockerignore
#5 transferring context: 2B done
#5 DONE 0.0s

#6 [internal] load build context
#6 transferring context: 555B done
#6 DONE 0.0s

#7 [1/5] FROM docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a36463b47
#7 resolve docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a36463b47
#7 resolve docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a36463b47 0.1s done
#7 DONE 0.1s

#8 [3/5] COPY requirements.txt .
#8 CACHED

#9 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#9 CACHED

#10 [2/5] WORKDIR /app
#10 CACHED
```

```
#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:7f7b74af1ec48348de1d9daf0b7d41655a6cd7fbf9f1ad63f489516e91d22dbe done
#12 exporting config sha256:f5b9fdb724d5dfdb15c2d66abd32241f26cf362e6c9d1565808e49e4f390fdc6 done
#12 exporting attestation manifest sha256:a6b8b4c8cfd722830b49859380df333a24f5b06f425b74e281e812364d0c2e20
#12 exporting attestation manifest sha256:a6b8b4c8cfd722830b49859380df333a24f5b06f425b74e281e812364d0c2e20 0.1s done
#12 exporting manifest list sha256:efc3e4dc8b6f52487a72492e1e735578785129073fb45b4ae68613890a9a7a8f 0.1s done
#12 naming to docker.io/library/byteforge_project-flask:latest 0.0s done
#12 unpacking to docker.io/library/byteforge_project-flask:latest 0.0s done
#12 DONE 0.3s

#13 resolving provenance for metadata file
#13 DONE 0.1s
[+] up 3/3
 ✓ Image byteforge_project-flask       Built                                   3.5s
 ✓ Container byteforge_project-flask-1 Recreated                               1.9s
 ✓ Container byteforge_project-nginx-1 Running                                 0.0s
Attaching to flask-1, nginx-1
flask-1  |  * Serving Flask app 'app'
flask-1  |  * Debug mode: off
flask-1  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
flask-1  |  * Running on all addresses (0.0.0.0)
flask-1  |  * Running on http://127.0.0.1:5000
flask-1  |  * Running on http://172.18.0.2:5000
flask-1  | Press CTRL+C to quit
flask-1  | 172.18.0.3 - - [05/Mar/2026 21:30:12] "GET / HTTP/1.0" 200 -
nginx-1  | 172.18.0.1 - - [05/Mar/2026:21:30:12 +0000] "GET / HTTP/1.1" 200 45 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWe
```



☐		●	mon-app-bytefc	5b91e9262b94	byteforge-a	5000:5000	0.02%	22.23MB / 3.71	
☐	›	●	byteforge_proje	-	-	-	0.02%	25.93MB / 7.42	

QUESTION DE REFLEXION

- **Pourquoi est-il dangereux d'exposer Flask directement sur Internet ?**

Le serveur intégré à flask est un serveur de développement.il n’est pas conçu pour supporter un grand nombre de connexions simultanées

En exposant flask directement mon conteneur est exposé aux attaques diverses

- **Comment Docker résout-il le nom 'flask' dans nginx.conf ?**

Docker dispose d’un système de résolution de noms DNS intégré au réseau bridge

Docker attribue automatiquement le nom d’hôte à l’adresse IP interne du conteneur flask ce qui permet le nginx de le trouver

- **Qu'apporte la séparation de réseau en termes de sécurité ?**

L'isolation: C’est à dire en plaçant flask et Nginx dans un réseau privé et en publiant que le port 80 de Nginx, flask devient invisible depuis mon réseau local

La limitation des mouvements : Si un conteneur est compromis, l’attaque est limité au réseau défini par docker, et il sera impossible d’avoir des attaques sur d'autres services qui ne partegeraient pas le même réseau

- **Que signifie :ro dans le volume ? Pourquoi c'est une bonne pratique ?**
- Le ro signifie read-only, il permet la protection de mon fichier de configuration lors d’une attaque malveillante.

Justification techniques

Choix technique	Conséquences de ne pas le faire	Faille évitée
L'utilisation de Nginx	Flask risque de ralentir et finir par s'arrêter si trop de gens se connectent en même temps	On évite que le serveur se plante
Cacher le port 5000	Attaque direct du réseau flask	Accès non autorisé et contournement des règles de sécurité proxy
Réseaux docker	Les services sont exposés sur l'IP de l'hôte par défaut	Exposition accidentelle de services sensible sur le réseau public